

# Computer Programming Summer Interest Group

---

Session 17 / Decorators

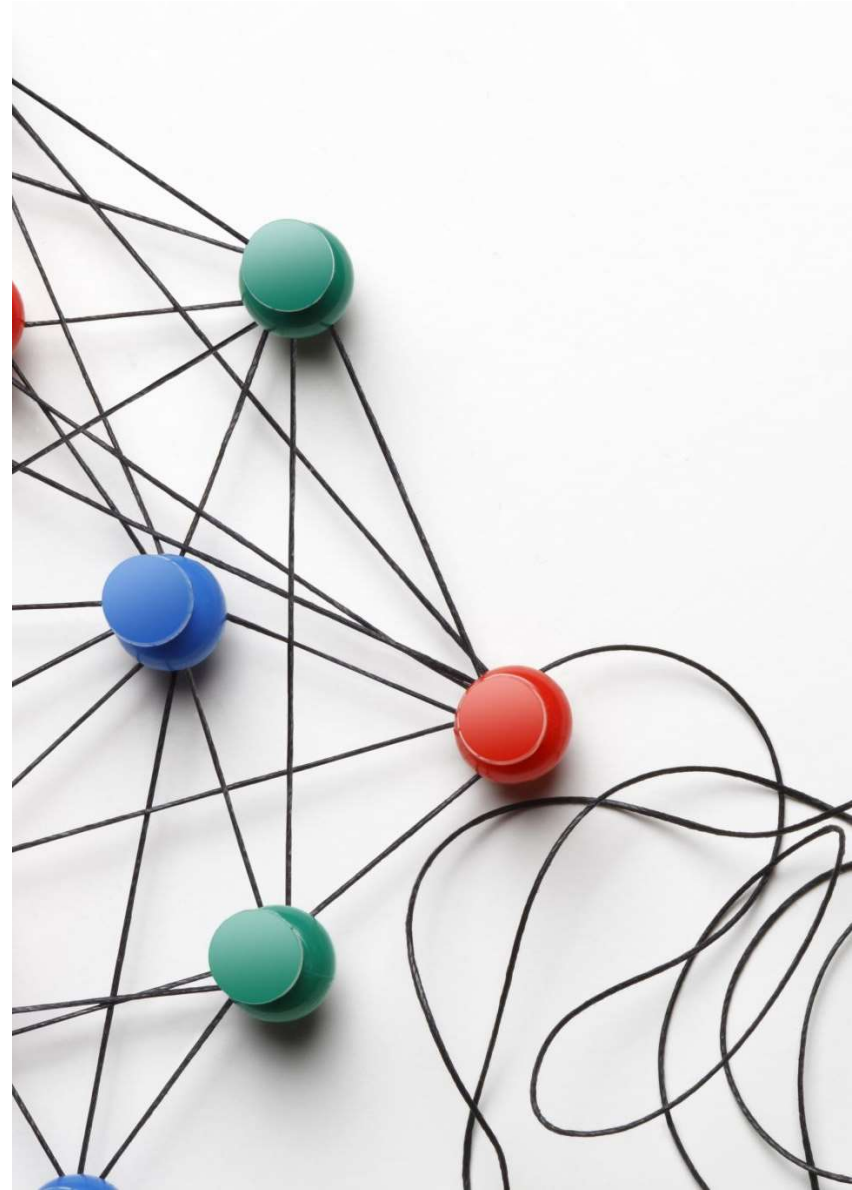
# 1 Peter 3:1-7

Likewise, wives, be subject to your own husbands, so that even if some do not obey the word, they may be won without a word by the conduct of their wives, when they see your respectful and pure conduct. Do not let your adorning be external—the braiding of hair and the putting on of gold jewelry, or the clothing you wear—but let your adorning be the hidden person of the heart with the imperishable beauty of a gentle and quiet spirit, which in God's sight is very precious. For this is how the holy women who hoped in God used to adorn themselves, by submitting to their own husbands, as Sarah obeyed Abraham, calling him lord. And you are her children, if you do good and do not fear anything that is frightening.

Likewise, husbands, live with your wives in an understanding way, showing honor to the woman as the weaker vessel, since they are heirs with you of the grace of life, so that your prayers may not be hindered.

# Objectives

- By the end of this session, you will be able to:
  - Understand what decorators are
  - Understand use of some common predefined decorators
  - Write simple function decorators
  - Write simple class decorators



# Decorators in Python

A decorator is a function which can be used to enhance the behavior or other functions or class methods

They can be applied to a single function or class method or to an entire class.

Python provides several built-in decorators, some of which we have previously seen (such as `@abstractmethod`)

# Some built-in Python Decorators

---

**@functools.lru\_cache** – Caches the “least recently used” return value for a given input value of a function to improve function performance

---

**@property** – Allows a method in a class to look like a read-only member of the class.

---

**@staticmethod** – Defines a class method which is bound to the class and not to any specific class instance and have no access to class or instance context

---

**@classmethod** – Similar to **@staticmethod** but retaining access to class (but not instance) context.

---

**@dataclasses.dataclass** – Defines certain methods of a class to simplify instantiation

---

**@abc.abstractmethod** – Specifies a method that must be defined in child class

---



# @functools.lru\_cache

- Using the recursive definition of the Fibonacci sequence in software can lead to making many extraneous computations.
- The use of **lru\_cache** can be used to cache the repetitive parts at a substantial speed-up.
- This uses the recursive approach one with caching and the other without
- The only difference is the decorator used to cache all intermediate results.
- The speed-up for the one using caching is considerable
- See sample\_17\_01.py for the full run

```
from functools import lru_cache
import time

def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

@lru_cache(maxsize=128)
def cached_fib(n: int) -> int:
    if n < 2:
        return n
    return cached_fib(n - 1) + cached_fib(n - 2)

start = time.perf_counter()
result = fib(35)
duration = (time.perf_counter() - start) * 1e-9
print(f"fib(35) = {result} / {duration} seconds")

start = time.perf_counter()
result = cached_fib(35)
duration = (time.perf_counter() - start) * 1e-9
print(f"cached_fib(35) = {result} / {duration} seconds")
```

# @property Decorators (getter / setter)

- The **@property** decorator creates an alias of a class method which returns a value associated with a class instance
- The value can be a just a class member or some value computed from one or more class members.
- A setter can be used to change a property previously defined
- The setter must immediately follow the property and have the same name and be decorated with the name of the property followed by **.setter**
- A “getter” property does not require the presence of a “setter”, but a “setter” must always accompany a “getter”.

```
import math

class Circle:
    def __init__(self, radius: float):
        self.radius = radius

    @property
    def area(self) -> float:
        return self.radius**2 * math.pi

    @area.setter
    def area(self, area: float):
        self.radius = math.sqrt(area / math.pi)

circle = Circle(radius=10)
print(f"{circle.radius=: .4f}\t{circle.area=: .4f}")

circle.area = 600
print(f"{circle.radius=: .4f}\t{circle.area=: .4f}")
```

# The @staticmethod Decorator

- The **@staticmethod** decorator designates methods associated with the class, but not a particular class instance
- Static methods have not contextual information regarding the class either (i.e., class name or the like)
- They do not take a **self** argument (which would be associated with specific class instance)
- They are logically equivalent to functions not associated with a class
- They can be called either by direct invocation using the class name (i.e. **SampleClass.set\_value**) or through an instance of the class (**sample.set\_value**).
- They can affect class attributes... in the example the class attribute is **value** and is not associated with a particular instance, but with the class as a whole. It will have the same value for all class instances

```
class SampleClass:
    value: int = 0

    @staticmethod
    def get_value() -> int:
        return SampleClass.value

    @staticmethod
    def set_value(value: int):
        SampleClass.value = value
```

```
sample = SampleClass()
print("\n**** value: 0 ****")
print(f"{sample.get_value()}=")
print(f"{SampleClass.get_value()}=")

SampleClass.set_value(value=10)
print("\n**** value: 10 ****")
print(f"{sample.get_value()}=")
print(f"{SampleClass.get_value()}=")

sample0.set_value(value=20)
print("\n**** value: 20 ****")
print(f"{sample.get_value()}=")
print(f"{SampleClass.get_value()}=")
```



## The `@dataclasses.dataclass` Decorator

- The **`@dataclasses.dataclass`** decorator allows an abbreviated specification of a class
- It is associated with a class and automatically defines the **`__init__`** method and some other attributes of the class
- It creates a simple class comprised of data elements that can be passed around as an object
- Methods can be associated with a dataclass, but this is generally not done... it is better to preserve the simplicity of the dataclass without additional methods.

```
from dataclasses import dataclass
import math
```

```
@dataclass
class Sample:
    key: str
    val: float
```

```
sample_pi = Sample(key="pi", val=math.pi)
sample_e = Sample(key="e", val=math.e)
sample_phi = Sample(
    key="phi",
    val=(1 + math.sqrt(5.0)) / 2.0)

print(f"{sample_pi.key=}\t{sample_pi.val=}")
print(f"{sample_e.key=}\t{sample_e.val=}")
print(f"{sample_phi.key=}\t{sample_phi.val=}")
```

# Custom Decorators

- It is possible to write your own decorators
- A decorator is a function that returns a function
- The returned function replaces the decorated function.
- In the example to right, calling **say\_hello** will print:

```
Before  
Hello  
After
```

- The downside is this overrides all of the decorated function's metadata

```
def my_decorator(func):  
    def wrapper():  
        """Wrapper"""  
        print("Before")  
        func()  
        print("After")
```

```
    return wrapper
```

```
@my_decorator  
def say_hello():  
    """Say hello"""  
    print("Hello")
```

```
say_hello()
```

```
print(f"    Name: {say_hello.__name__}")  
print(f"    Docstring:  
{say_hello.__doc__}")
```

# Custom decorator with `@functools.wraps`

- Using `@functools.wraps` will preserve the function's metadata
- We apply the `@wraps` decorator to the wrapper function we return to preserve the decorated function metadata
- Also note the arguments for the wrapper function
- **args** is a list of positional arguments to pass to wrapper
- **kwargs** is a dictionary of keyword arguments to pass to the function
- This allows wrapper to take an arbitrary collection of positional and keyword arguments and pass them to the decorated function

```
import time
from functools import wraps
from typing import Any, Callable

def time_it(func: Callable) -> Callable:
    @wraps(func)
    def wrapper(*args, **kwargs) -> Any:
        """wrapper docstring"""
        start = time.perf_counter()
        result = func(*args, **kwargs)
        duration = time.perf_counter() - start_time
        print(f'"{func.__name__}" executed in {duration:.4f} sec')
        return result

    return wrapper

@time_it
def sleep(delay: float) -> str:
    """Sleep docstring"""
    time.sleep(delay)

sleeper(0.2)
sleeper(1.5)

print(f"Name: {sleep.__name__}")
print(f"Docstring: {sleep.__doc__}")
```

# Custom class decorator

- We can also write class decorators, but it's a little more complicated.
- This example applies logging to all user defined methods of a class to track when a method is called and with what arguments and what return values.
- In this example we need two decorators, one decorator for the class, and another decorator the class decorator applies to all of the user defined methods in the class.
- The **for** loop in **log\_all\_methods** loops over all of the attributes of the class type, filtering out all but the user defined methods (in this case **add** and **hello**)
- The **setattr** function call in the loop effectively applies the **log\_method** decorator to each of the user defined methods in the class.
- We're using **functools.wraps** to preserve the metadata of the wrapped methods.
- See sample\_17\_07.py for a slightly expanded example

```
from functools import wraps
from typing import Callable, Type

def log_method(func: Callable):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Enter: {func.__qualname__}, {args=}, {kwargs=}")
        result = func(*args, **kwargs)
        print(f"Exiting: {func.__qualname__} (returned {result!r})")
        return result

    return wrapper

def log_all_methods(cls: Type) -> Type:
    for attr_name, attr_value in cls.__dict__.items():
        if callable(attr_value) and not attr_name.startswith("__"):
            setattr(cls, attr_name, log_method(attr_value))
    return cls

@log_all_methods
class Calculator:
    def add(self, a: float, b: float) -> float:
        return a + b

    def hello(self, name: str):
        print(f"Hello, {name}!")

calc = Calculator()
calc.add(10, 5)
calc.hello(name="Alice")
```



# Activities

- We can use decorators to profile a class by counting the number of times a method is called and recording the time each call lasts in a list.
- Write a class decorator that records the amount of time spent each time a class method is called.
- Store the profile data in a dictionary with the key being the qualified name of the method and the value a list of times spent in the method.
- Create a class containing the two Fibonacci computations (sample\_17\_01.py) and profile each.